

# Extensiones de Machine Learning: Aprendizaje por Refuerzo

Miguel Ángel Ortiz Peñaranda  
`miguelangel.ortiz1@um.es`

17 de junio de 2026

## Resumen

En este documento se va a presentar el análisis y la experimentación correspondiente a la primera parte de la asignatura Extensiones de Machine Learning del Máster en Inteligencia Artificial, que versa sobre aprendizaje por refuerzo, rama de la IA que estudia cómo un agente aprende a tomar decisiones mientras interactúa con un entorno determinado.

Esta materia la componen dos partes fundamentales. La primera parte se corresponde con el **Bandido de K-Brazos**, problema que trata sobre cómo maximizar una recompensa mediante la toma de decisiones en un entorno donde existen varias opciones, o "brazos", estableciendo un consenso entre exploración de nuevos brazos y explotación de uno en particular. Se estudiará diversos tipos de bandido y algoritmos que maximizan la recompensa, y evaluaremos su desempeño en distintos experimentos.

La segunda parte es aquella correspondiente al **Aprendizaje en Entornos Complejos**, donde se estudiará diversos métodos que permiten tomar decisiones en situaciones donde el número de estados y acciones posibles es significativo. Se indagará en dos tipos de métodos para este cometido, los **métodos tabulares**, que guardan los valores para cada estado-acción, y los **métodos aproximados**, que, en lugar de almacenarlo todo en tablas, utilizan algunas técnicas que les permiten estimar estos valores. Para cada método, se realizará una comparativa y trataremos de determinar cuál consigue ofrecer mejores resultados.

# Índice general

|                                                                 |          |
|-----------------------------------------------------------------|----------|
| <b>1. Problema del Bandido de <math>k</math>-brazos</b>         | <b>3</b> |
| 1.1. Introducción . . . . .                                     | 3        |
| 1.2. Desarrollo . . . . .                                       | 3        |
| 1.2.1. Definición formal del bandido multibrazo . . . . .       | 3        |
| 1.2.2. Tipos de bandido multibrazo . . . . .                    | 3        |
| 1.2.3. Trabajos Relacionados . . . . .                          | 4        |
| 1.3. Algoritmos . . . . .                                       | 4        |
| 1.3.1. Epsilon-Greedy . . . . .                                 | 4        |
| 1.3.2. Upper Confidence Bound (UCB-1) . . . . .                 | 4        |
| 1.3.3. Softmax . . . . .                                        | 5        |
| 1.4. Evaluación/Experimentos . . . . .                          | 5        |
| 1.4.1. Descripción de los experimentos . . . . .                | 5        |
| 1.4.2. Resultados . . . . .                                     | 5        |
| 1.5. Conclusiones . . . . .                                     | 6        |
| <b>2. Aprendizaje en Entornos Complejos</b>                     | <b>7</b> |
| 2.1. Introducción . . . . .                                     | 7        |
| 2.2. Técnicas Tabulares de Aprendizaje por Refuerzo . . . . .   | 7        |
| 2.2.1. Algoritmos . . . . .                                     | 8        |
| 2.2.2. Evaluación/Experimentos . . . . .                        | 8        |
| 2.2.3. Conclusiones . . . . .                                   | 10       |
| 2.2.4. Vías futuras . . . . .                                   | 10       |
| 2.3. Técnicas Aproximadas de Aprendizaje por Refuerzo . . . . . | 10       |
| 2.3.1. Algoritmos . . . . .                                     | 11       |
| 2.3.2. Wrappers del entorno . . . . .                           | 11       |
| 2.3.3. Evaluación/Experimentos . . . . .                        | 11       |
| 2.3.4. Conclusiones . . . . .                                   | 12       |

2.3.5. Vías futuras . . . . . 13

# Capítulo 1

## Problema del Bandido de $k$ -brazos

### 1.1. Introducción

El problema del bandido multibrazo trata de, dado un número determinado y finito ( $k$ ) de opciones, o *brazos*, tratar de seleccionar aquel que consigue en cada momento maximizar la recompensa acumulada, es decir, encontrar aquel brazo que de un mayor número de recompensas altas en el menor tiempo posible, siendo las recompensas otorgadas por cada brazo totalmente desconocidas a priori.

Este problema fue formalizado por Herbert Robbins en el año 1952, en su artículo "Some aspects of the sequential design of experiments", donde se plantea la versión clásica del problema, ilustrada con máquinas tragaperras (*bandits*), y se introduce también la importancia en este ámbito de los conceptos **exploración** y **explotación**.

Este problema goza de especial relevancia debido fundamentalmente a esta introducción del consenso entre exploración y explotación, que fija la capacidad de un algoritmo para "explorar" nuevos brazos, o estancarse en uno determinado, el cual espera consiga maximizar la recompensa, la introducción del concepto de "rechazo" para evaluar el algoritmo, mediante la comparación de los resultados obtenidos con los ideales (para esto se debe poder conocer a priori el brazo óptimo), y porque supone una problema tipo en el contexto del aprendizaje por refuerzo.

En este informe se dará una descripción formal del algoritmo y de algunas variantes del mismo. Se estudiarán distintas estrategias para maximizar la recompensa, y se establecerá, por medio de una experimentación, una comparativa entre las mismas, de forma que consigamos determinar cuáles ofrecen mejor desempeño.

### 1.2. Desarrollo

#### 1.2.1. Definición formal del bandido multibrazo

El problema del bandido de  $k$ -brazos se puede definir formalmente de la siguiente forma:

1. Tenemos un conjunto  $A$  de  $K$  acciones:

$$A = \{a_1, a_2, a_3, \dots, a_k\}$$

2. Tenemos también un conjunto de pasos de tiempo para los que se evalúa el bandido:

$$T = 1, 2, 3, \dots, T$$

3. En cada paso de tiempo:

- Elegimos uno de los brazos (acciones):  $a_t \in A$
- Se obtiene una recompensa en base a esa acción, dada una distribución desconocida:  $r_t \sim P_r(r|a_t)$

4. Acumulamos esa recompensa

$$\sum_{t=1}^T r_t$$

5. El objetivo para cualquier variante del bandido será el de maximizar esa recompensa acumulada

#### 1.2.2. Tipos de bandido multibrazo

Esencialmente, se han descrito tres tipos de bandido multibrazo:

- **Bandido Estocástico:** Asume que cada brazo tiene una distribución de recompensas fija pero desconocida. Cada vez que se elige un brazo, se recibe una recompensa aleatoria generada por esa distribución, y el objetivo es aprender cuál tiene la mejor media esperada. El entorno es estable y no cambia con el tiempo.
- **Bandido Adversarial:** No presupone ninguna estructura probabilística en las recompensas. El entorno puede cambiar arbitrariamente e incluso comportarse de manera hostil para perjudicar al algoritmo. Aquí no existe necesariamente un mejor brazo fijo, así que el objetivo es obtener un rendimiento cercano al mejor brazo observado hasta la fecha.
- **Bandido Contextual:** Incorporan información adicional antes de tomar una decisión, a la que se denomina contexto. La recompensa de cada brazo depende tanto del brazo elegido como del contexto observado, por lo que no existe un mejor brazo universal. El objetivo es aprender qué acción funciona mejor para cada situación concreta.

Este documento se va a centrar en el estudio del **bandido estocástico**, y no indagaremos en los otros dos tipos.

En el bandido estocástico, a la hora de seleccionar un brazo, como se ha descrito anteriormente, este brazo devuelve una recompensa de forma acorde a una distribución de probabilidad a priori desconocida  $r \sim P_a$ . El documento se va a centrar en la representación del bandido siguiendo las siguientes distribuciones:

- **Distribución binomial:** La distribución binomial modela el número de ocurrencias de un evento determinado

(obtener una recompensa), bajo una probabilidad  $p$ . En este caso, dada una serie de acciones,  $n$ , se obtendrá una recompensa con probabilidad  $p$  en cada una de ellas. Esto puede ser representado por una tupla donde se asocian ambos términos, como  $\beta(n, p)$ .

- ▶ **Distribución de Bernoulli:** Este tipo de distribución es un tipo particular de distribución binomial, donde  $n = 1$ . En este caso, en lugar de realizarse  $n$  acciones (y por tanto, existir  $n$  posibilidades de acierto/error), tan solo se contempla la posibilidad de un único éxito o fracaso.
- ▶ **Distribución normal:** La distribución normal no contabiliza éxitos o fracasos, sino que queda representada por variables continuas, dentro de un dominio concreto. En este caso, la recompensa se corresponderá con el valor discreto aportado por el brazo cuando se interacciona con el.

En cada caso, la expresión “maximizar la recompensa total obtenida”, tiene un significado distinto **dependiendo de la distribución** que represente cada problema del bandido multibrazo. Por ejemplo, en el caso de una distribución normal, es posible que nos interese maximizar el valor total acumulado tras realizar cada acción. En el caso de la distribución binomial o de bernoulli, nos interesará maximizar el número de aciertos. Cada tipo de bandido multibrazo tiene diversas aplicaciones en el mundo real, como la contabilización de accesos a contenido publicitario (distribución de Bernoulli), o para tratamientos médicos (una distribución normal, para cada tratamiento, sopesa distintos factores, como recuperación, dolor, etc.).

Para lograr este fin, existen distintos criterios, para seleccionar, en cada momento, una acción:

- ▶ Un enfoque bastante intuitivo puede ser el de “instar” al algoritmo a ceñirse a los brazos, a las acciones, que históricamente han aportado **recompensas mayores**.
- ▶ Otro enfoque puede ser el de cuantificar la pérdida si comparamos con los resultados “ideales” que hubiéramos obtenido eligiendo siempre el mejor brazo, en caso de conocer cuál es el mismo. Normalmente en la experimentación es autor quien implementa el test y es por tanto capaz de conocer esto último. A esta cuantificación se la denomina **rechazo**.

En los experimentos desarrollados, se ha elegido el primer enfoque por haberse considerado el más **intuitivo** de los dos, si bien son igual de válidos.

Históricamente se han estudiado multitud de métodos para dar una resolución al problema del bandido multibrazo, desde los más simples, como el *Round Robin*, donde sencillamente se itera por todos los brazos, explotando cada uno sin tener en cuenta la recompensa obtenida, como los más sofisticados, que aprenden de las recompensas que se han obtenido a lo largo del tiempo y utilizan esa información para intentar predecir qué brazo es el idóneo. Recordemos que todos los métodos son *estimativos* y ninguno dispone de información a priori sobre el desempeño de cada brazo.

Este estudio tiene como objetivo comparar algunos de estos métodos, y determinar cuáles obtienen mejores resultados en el problema del bandido, en condiciones similares. Para ello,

dispondremos de distintas variantes del problema del bandido (distribución normal, binomial y de bernoulli), las cuales analizaremos por separado, para también poder probar si existe un método aparentemente “universal”, o por el contrario, dependiendo del tipo de problema, algunos presentan mejores resultados que otros.

### 1.2.3. Trabajos Relacionados

Otros tipos de bandido han sido tratados más en profundidad en trabajos como *Contextual Bandits: Foundations and Applications* [1].

## 1.3. Algoritmos

Una vez definido el problema del bandido, se va a describir algunas estrategias que llevan la maximización de esta recompensa que hemos descrito. Si bien existen numerosos métodos de distinta complejidad que persiguen este objetivo, todos suelen tener un mismo eje vertebral común: El establecimiento de un consenso entre exploración de distintos brazos, o explotación de uno en particular.

Este documento se va a centrar en analizar el funcionamiento y establecer una comparativa entre los siguientes tres métodos, que determinan en qué medida y de qué forma se realiza la labor de exploración.

### 1.3.1. Epsilon-Greedy

Epsilon Greedy elige la mejor acción conocida (la que mayor recompensa acumulada nos ha aportado hasta la fecha), pero con una probabilidad  $\epsilon$  se explora una acción aleatoria. P.ej, si  $\epsilon = 0$ , el algoritmo se estancará únicamente en la mejor acción conocida, privándose de descubrir otras que pudieran ser potencialmente más ventajosas. Si  $\epsilon = 0,1$ , el algoritmo exploraría otros brazos con una probabilidad del 10%, de forma que podría encontrar otro cuya recompensa resultara ser mayor.

### 1.3.2. Upper Confidence Bound (UCB-1)

Este algoritmo selecciona el brazo combinando dos factores:

- ▶ La **recompensa** media observada hasta la fecha
- ▶ La **incertidumbre** asociada a cada brazo

La idea es favorecer no solo los brazos que han dado buenas recompensas, sino también aquellos que se han sido probados “poco” y podrían resultar mejores en potencia. Para ello añade un término que disminuye a medida que un brazo se explora más veces. Así se consigue equilibrar automáticamente exploración y explotación. Se rige por la siguiente relación:

$$A_t = \operatorname{argmax}_{a \in A} Q_t(a) + c \sqrt{\frac{2 \ln t}{N_t(a)}}$$

Donde el parámetro  $c$  se corresponde con un parámetro de ajuste que regula el grado de exploración que realiza el algoritmo, de forma que un  $c = 0$  se comportaría como un algoritmo greedy, anulando la segunda parte de la ecuación y

quedándose solo con la recompensa acumulada para esa acción.

### 1.3.3. Softmax

Asigna a cada brazo una probabilidad de ser elegido en función de su recompensa media estimada. En lugar de escoger siempre el mejor brazo conocido, transforma esas estimaciones en probabilidades mediante una función exponencial:

$$P(A_t = a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^k e^{Q_t(b)/\tau}}$$

Donde el parámetro de temperatura,  $\tau$ , controla la componente exploratoria. A valores más grandes, explora más otros brazos. De esta forma, los brazos con mejores recompensas tienen más opciones de ser seleccionados, pero los demás aún pueden ser explorados ocasionalmente.

## 1.4. Evaluación/Experimentos

### 1.4.1. Descripción de los experimentos

Se ha realizado una serie de experimentos de cara a establecer una comparativa entre los tres algoritmos descritos. Para estos experimentos, se han utilizado Notebooks de Python en aras de favorecer la portabilidad y la simplicidad del código.

En estos Notebooks se ha implementado el propio problema del bandido multibrazo, que queda conformado por un conjunto parametrizable de  $K$  Brazos, cada uno con una distribución de las que hemos mencionado anteriormente. Las distribuciones que se mueven en un rango de valores, como la normal, quedan parametrizadas por ese valor máximo de recompensa. Se conoce a priori el mejor brazo para esa instancia del bandido.

Se ha implementado también los tres algoritmos que hemos descrito que nos permiten buscar una solución al problema del bandido (**Epsilon-Greedy**, **UCB-1** y **Softmax**). Cada algoritmo queda parametrizado, en su caso, por el valor del parámetro de exploración (o de temperatura, en su caso).

Vamos a utilizar el **porcentaje de elecciones óptimas en el último 20 % de pasos** para evaluar qué algoritmo acaba obteniendo un mejor desempeño, comparando con el mejor brazo. El porqué utilizar sólo el tramo final de su evaluación es porque consideramos, es un período donde, a esas alturas, el algoritmo puede haberse estabilizado ya (haber elegido un brazo potencialmente “mejor”). Queremos por tanto no tener en cuenta posibles errores cometidos cuando el algoritmo todavía estaba “aprendiendo”. Siendo así, el algoritmo con mayor porcentaje final de aciertos, será considerado el que mejor funciona para esa configuración, en esa distribución.

Partiendo de esta base, se han desarrollado tres notebooks distintos, que comparan distintas instancias de esos algoritmos, para el mismo tipo de distribución. A cada notebook se lo considera un experimento independiente:

- ▶ Bandido Multibrazo para distribución **normal**
- ▶ Bandido Multibrazo para distribución **binomial**

- ▶ Bandido Multibrazo para distribución de **Bernoulli**

En cada notebook, se ha ejecutado el problema para **10 y 20 brazos (K)**, entendiendo que un número superior de brazos implica una mayor dificultad para obtener una solución deseable. Para cada número de brazos, también se ha probado con **1000 y 2000 pasos (T)** (steps), de cara a ver si un tiempo de ejecución más prolongado favorece a alguno de los algoritmos mencionados. A cada configuración de brazos la denominamos una nueva “variante del bandido”.

Para el notebook con distribución normal, también se ha realizado dos pruebas mas con  $K=20$ , steps=2000, pero probando con valores de **desviación estándar de 5 y 10** (en las pruebas anteriores era de 1), para comprobar si una mayor desviación interfiere en el funcionamiento de los algoritmos. Hemos considerado también esta modificación como otra variante del bandido.

### 1.4.2. Resultados

A continuación se presentan los resultados en cuanto a número de aciertos para cada distribución. Se ha promediado el valor obtenido en los dos experimentos con número distinto de pasos, y distintos valores de desviación, en su caso, de cara a dar una visión más general de los mismos.

Se exponen en primer lugar los resultados de las tres variantes del bandido para la distribución normal. Estas variantes son las siguientes:

- ▶ V1 - Experimento para bandido con  $K=10$  (promedio de 1000 y 2000 steps)
- ▶ V2 - Experimento para bandido con  $K=20$  (promedio de 1000 y 2000 steps)
- ▶ V3 - Experimento para bandido con  $K=20$ , 2000 steps (promedio de desviación 5 y 10)

Tabla 1.1: Resultados para distribución normal

| Algoritmo                | V1             | V2             | V3             | Promedio       |
|--------------------------|----------------|----------------|----------------|----------------|
| EG ( $\epsilon = 0$ )    | 82.75 %        | 96.75 %        | 51.25 %        | 76.92 %        |
| EG ( $\epsilon = 0,01$ ) | 65.07 %        | 46.66 %        | 32.91 %        | 48.21 %        |
| EG ( $\epsilon = 0,1$ )  | 89.85 %        | 89.04 %        | 67.75 %        | 80.88 %        |
| UCB ( $c = 0,5$ )        | <b>99.39 %</b> | <b>99.96 %</b> | 50.50 %        | 83.28 %        |
| UCB ( $c = 1$ )          | 99.35 %        | 99.67 %        | 60.87 %        | 86.63 %        |
| UCB ( $c = 2$ )          | 96.77 %        | 98.39 %        | <b>69.00 %</b> | <b>88.05 %</b> |
| SM ( $\tau = 0,5$ )      | 13.93 %        | 9.56 %         | 4.00 %         | 9.17 %         |
| SM ( $\tau = 1$ )        | 33.23 %        | 34.27 %        | 5.00 %         | 24.17 %        |
| SM ( $\tau = 2$ )        | 28.05 %        | 27.75 %        | 5.50 %         | 20.43 %        |

A continuación figuran resultados de las dos variantes del bandido para la distribución binomial (no existe la variante con distinta desviación en este caso).

- ▶ V1 - Experimento para bandido con  $K=10$  (promedio de 1000 y 2000 steps)
- ▶ V2 - Experimento para bandido con  $K=20$  (promedio de 1000 y 2000 steps)

Por último se pueden ver los resultados de las dos variantes del bandido para la distribución de Bernoulli.

Tabla 1.2: Resultados para distribución binomial

| Algoritmo                | V1             | V2             | Promedio       |
|--------------------------|----------------|----------------|----------------|
| EG ( $\epsilon = 0$ )    | 84.25 %        | 69.75 %        | 77.00 %        |
| EG ( $\epsilon = 0,01$ ) | 68.21 %        | 37.75 %        | 52.98 %        |
| EG ( $\epsilon = 0,1$ )  | 90.32 %        | 83.51 %        | 86.91 %        |
| UCB-1 ( $c = 0,5$ )      | 97.16 %        | 95.41 %        | 96.28 %        |
| UCB-1 ( $c = 1$ )        | <b>99.15 %</b> | <b>98.39 %</b> | <b>98.77 %</b> |
| UCB-1 ( $c = 2$ )        | 96.39 %        | 90.60 %        | 93.50 %        |
| Softmax ( $\tau = 0,5$ ) | 14.77 %        | 6.80 %         | 10.78 %        |
| Softmax ( $\tau = 1$ )   | 40.72 %        | 19.45 %        | 30.08 %        |
| Softmax ( $\tau = 2$ )   | 26.85 %        | 14.11 %        | 20.48 %        |

- V1 - Experimento para bandido con  $K=10$  (promedio de 1000 y 2000 steps)
- V2 - Experimento para bandido con  $K=20$  (promedio de 1000 y 2000 steps)

Tabla 1.3: Resultados para distribución binomial

| Algoritmo                | V1             | V2             | Promedio       |
|--------------------------|----------------|----------------|----------------|
| EG ( $\epsilon = 0$ )    | 60.00 %        | 56.69 %        | 58.34 %        |
| EG ( $\epsilon = 0,01$ ) | 52.78 %        | 25.54 %        | 39.16 %        |
| EG ( $\epsilon = 0,1$ )  | 75.10 %        | <b>63.84 %</b> | <b>69.47 %</b> |
| UCB-1 ( $c = 0,5$ )      | <b>78.00 %</b> | 52.99 %        | 65.50 %        |
| UCB-1 ( $c = 1$ )        | 47.30 %        | 21.30 %        | 34.30 %        |
| UCB-1 ( $c = 2$ )        | 26.25 %        | 10.92 %        | 18.58 %        |
| Softmax ( $\tau = 0,5$ ) | 16.21 %        | 8.21 %         | 12.21 %        |
| Softmax ( $\tau = 1$ )   | 12.91 %        | 6.49 %         | 9.70 %         |
| Softmax ( $\tau = 2$ )   | 11.46 %        | 5.62 %         | 8.54 %         |

## 1.5. Conclusiones

- Estos algoritmos han demostrado ser efectivos para la aproximación a una solución deseable del problema del bandido. No existe una solución ni parámetros universales, sino que, dependiendo del tipo de distribución, quizás nos interese una u otra.
- En términos generales, UCB-1 con  $c=0.5$  y Epsilon-Greedy con un valor moderado para epsilon, como 0.1, han obtenido un desempeño satisfactorio.
- Softmax no ha tenido un buen desempeño, probablemente causado por no haber invertido tiempo en elegir unos parámetros de temperatura más acordes a los problemas a tratar.
- En variantes del bandido más complejas, como son la desitribución de Bernoulli, existe más ruido y menos información para que cada algoritmo pueda determinar qué brazo es el mejor. Se requiere, por tanto, un número de generaciones significativamente mayor para llegar a resultados suficientemente buenos.

# Capítulo 2

## Aprendizaje en Entornos Complejos

### 2.1. Introducción

Este capítulo se orquestará en torno al tema de Aprendizaje en Entornos Complejos, proceso mediante el cual un agente interactúa con un entorno determinado, que puede ser estático o cambiante, y aprende a tomar decisiones en base a las observaciones que realiza sobre ese entorno. En particular, este estudio se va a centrar en entornos estáticos.

Este tipo de problemas suele modelarse mediante Procesos de Decisión de Markov, que describen los estados existentes en el entorno ( $S$ ), las posibles acciones que se pueden realizar ( $A$ ), las potenciales recompensas ( $R$ ) y las transiciones entre estados ( $P$ ). Sobre este marco teórico se construyen las funciones de valor y las ecuaciones de Bellman, que establecen una relación entre el valor de un estado determinado y los valores esperados de los estados futuros. De esta forma, el valor esperado de, dado un estado  $s$ , realizar la acción  $a$ , siguiendo una política  $\pi$ , se podría expresar de la siguiente forma:

$$q_{\pi}(s, a) = E_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]$$

Donde, como vemos, se asocia el valor esperado del siguiente par estado-acción que se alcanza tras la transición, con la recompensa inmediata que se obtiene, dado un estado actual, y utilizando un factor de descuento del futuro, identificado como  $\gamma$ .

La política, identificada como  $\pi$ , define la estrategia que sigue el agente al interactuar con el entorno. Dicho de otra forma, es la componente que dictamina, dado un estado determinado, cuál es la siguiente acción a realizar por parte del agente. Esta política es la que interesa al agente ir aprendiendo y perfeccionando durante su interacción. Existen dos paradigmas según se disponga este aprendizaje de la política:

- ▶ *On-Policy*: La política que dicta las acciones del agente es la misma que se va evaluando y perfeccionando durante su aprendizaje.
- ▶ *Off-Policy*: El agente aprende una política objetivo (*target policy*), que no es necesariamente la misma que utiliza a la hora de elegir la acción a realizar (*behavior policy*).

Esta distinción es importante porque los agentes pueden utilizar un enfoque u otro, y esto determina en gran medida su funcionamiento.

En cuanto al retorno esperado de un estado-acción, en la mayoría de los casos, es desconocido a priori, por lo que el agente

debe estimarlo a partir de su experiencia. Para ello, existen diversos algoritmos de aprendizaje, que consiguen, dada la experiencia conseguida interactuando con el entorno, generar conocimiento y “aprender” del mismo. Estos algoritmos pueden ser distinguidos en dos tipos fundamentales:

- ▶ **Métodos Tabulares**: Son algoritmos cuyo agente almacena una tabla  $q(s, a)$ , donde guarda el retorno esperado para cada par estado-acción. El agente va modificando esta tabla conforme va adquiriendo experiencia sobre el entorno, y la va utilizando también para tomar las decisiones, de forma que, en un estado concreto, elegirá generalmente la acción que maximice el retorno esperado.
- ▶ **Métodos Aproximados**: En entornos más complejos, como aquellos que presentan un espacio de estados o acciones continuos, es inviable almacenar una tabla para cada combinación de estado-acción. Estos métodos, por lo tanto, construyen una función que logre aproximar ese retorno esperado. De esta forma, para cada estado se puede calcular una aproximación del retorno esperado para cada acción.

Detallaremos estos dos tipos de métodos en las secciones venideras.

### 2.2. Técnicas Tabulares de Aprendizaje por Refuerzo

El modelo tabular almacena una tabla  $Q(s, a)$ , donde se indica, para cada posible estado de un problema y cada posible acción a realizar, el valor esperado del retorno (recompensa) de la misma. De esta forma, durante la interacción con el entorno, el agente es entrenado para elegir la acción más ventajosa dependiendo del estado en el que se encuentre, siguiendo una política determinada.

Los modelos tabulares presentan una serie de ventajas claras:

- ▶ Son sencillos de implementar y de comprender, tanto a nivel de funcionamiento como a nivel de resultados.
- ▶ Existen garantías de que los modelos acaben convergiendo hacia una política óptima.
- ▶ Para entornos discretos presentan un buen desempeño si son configurados correctamente. Su entrenamiento es también por lo general bastante rápido, pues acceder a cada celda es inmediato.

Sin embargo, traen consigo también una serie de desventajas

inherentes a su funcionamiento:

- ▶ Al almacenar una matriz bidimensional por cada par estado-acción, tenemos que esta tabla crece desproporcionadamente cuando el número de estados o acciones posibles es grande. Esto puede acabar suponiendo un problema de memoria cuando la tabla es significativa.
- ▶ Estos agentes funcionan bien para conjuntos de estados y acciones discretos, pero son incapaces de resolver problemas cuyos conjuntos sean continuos, pues son valores para los que no es posible confeccionar una tabla.
- ▶ No son capaces de generalizar. Aprenden el valor para un estado concreto, pero no tienen la capacidad de extrapolar ese aprendizaje a estados que sean similares.

Este estudio se ha centrado en el entrenamiento y comparativa de los modelos tabulares llamados Monte Carlo On-Policy, Monte Carlo Off-Policy, SARSA y Q-Learning.

### 2.2.1. Algoritmos

A continuación se reseñará brevemente los distintos métodos de aprendizaje valorados en este estudio. En aras de reducir la magnitud del documento, no se va a exponer el pseudocódigo asociado a cada algoritmo.

#### Monte Carlo On-Policy

Monte Carlo On-Policy estima los valores de los pares estado-acción utilizando episodios completos generados por la misma política que se desea aprender. Las actualizaciones se realizan al finalizar cada episodio a partir de los retornos observados, de forma que se itera en orden inverso y se actualiza su tabla  $q(s, a)$ .

#### Monte Carlo Off-Policy

Monte Carlo Off-Policy funciona de forma similar a Monte Carlo, pero permite aprender una política objetivo distinta de la política utilizada para generar las experiencias. Para ello emplea técnicas como muestreo por importancia, que asigna un peso a cada episodio según lo probable que habría sido observarlo bajo la política objetivo. De esta forma, se corrigen las diferencias entre ambas políticas durante la estimación de los retornos.

#### SARSA

SARSA es un método de Diferencias Temporales (*Temporal Differences*) y aprendizaje *on-policy* que actualiza el valor de un par estado-acción utilizando la recompensa obtenida y el valor estimado de la siguiente acción realmente seleccionada por la política actual. Esta actualización se realiza en cada nuevo paso de tiempo, y no es necesario, a diferencia de Monte Carlo, que termine un episodio completo.

#### Q-Learning

Q-Learning es un método de Diferencias Temporales y aprendizaje *off-policy* que actualiza los valores de su tabla utilizando la mejor acción posible (política *greedy*) en el siguiente estado, independientemente de la acción que el agente haya ejecutado realmente durante la exploración.

Todos los métodos funcionarán de acuerdo a una política  $\epsilon$ -soft, que significa sencillamente que todas las acciones tendrán una probabilidad distinta de 0 de ser elegidas. De esta forma, el agente, con una probabilidad  $1-\epsilon$  elegirá la mejor acción que dicte su tabla  $q(s, a)$ , y con una probabilidad  $\epsilon$  elegirá una acción aleatoria, de cara a explorar nuevas acciones.

Se utilizará también  $\epsilon$ -decaimiento en algunos experimentos, que implica que el valor de  $\epsilon$  va decreciendo conforme al transcurso de los episodios, de forma que la política va dejando de explorar y va explotando más las mejores acciones conocidas, paulatinamente.

## 2.2.2. Evaluación/Experimentos

### Descripción de los experimentos

El objetivo de la experimentación para métodos tabulares ha sido el de establecer una comparativa entre los distintos tipos de agente que hemos comentado, utilizando distintos hiperparámetros, de cara a evaluar el desempeño de todos ellos utilizando la API de *gymnasium* (<https://gymnasium.farama.org/index.html>), que permite simular un entorno de aprendizaje, y ofrece diversas configuraciones y problemas predefinidos que simplifican la labor. Estos entornos ya tienen su propia configuración de estados, acciones y recompensas. De esta forma, se ha podido abstraer este estudio de la implementación del entorno para centrarse únicamente en la implementación de los propios agentes. Los entornos escogidos de *gymnasium* han sido los siguientes:

- ▶ **Frozen Lake:** Trata de cruzar un mapa de celdas, desde un origen hasta una posición objetivo, intentando no caer en uno de los agujeros que existen en el lago. Existen tantos estados como casillas posibles, y tantas acciones como movimiento en una dirección posibles (arriba, abajo, izquierda, derecha). La recompensa será de 1 al llegar al objetivo y 0 en cualquier otro caso. El episodio termina si se cae por un agujero, se llega al objetivo, o se trunca por tiempo.
- ▶ **Taxi:** Trata de guiar un agente a lo largo y ancho de un mapa de celdas, de forma que recoja a un pasajero en una de las cuatro celdas previamente designadas (es aleatorio), y lo deposite en otra de las cuatro celdas. El conjunto de estados lo componen las posibles posiciones del taxi, del pasajero, y donde depositarlo, y el conjunto de acciones, adicionalmente a los cuatro posibles movimientos, recoger y dejar pasajero. Las recompensas son de -1 por cada paso por defecto, +20 por dejar a un pasajero, y -10 por realizar acciones no permitidas (intentar dejar a un pasajero sin haber recogido a uno p.ej.).

En cuanto a la metodología de trabajo, se va a entrenar, tanto con distintas variantes del entorno, en caso de haberlas, como con distintas variantes de los agentes tabulares y diversos hiperparámetros para los mismos. Siendo así, seguiremos los siguientes pasos:

1. Para cada entorno, entrenaremos todos los tipos de agentes tabulares comprendidos, utilizando un grid de hiperparámetros. El entrenamiento se realizará **a lo largo de 50000** episodios para cada combinación.

2. Se obtendrán una serie de estadísticas que nos permitan graficar la recompensa obtenida por cada agente a lo largo del tiempo, y la evolución de la longitud de cada episodio, en *steps*.
3. Se comparará la mejor recompensa obtenida por el agente con el resto de agentes, y la mejor longitud de episodio que es capaz de conseguir con la del resto de agentes y con la óptima.
4. Se pomediarán los resultados obtenidos para cada entorno, para cada agente.
5. Se darán unas conclusiones finales que indiquen qué algoritmo o algoritmos se considera que presentan un mejor comportamiento.

En cuanto al grid de parámetros, evaluaremos cada variante del entorno primero con un grid donde constan distintos valores para un epsilon fijo, y valores para el factor de descuento del retorno futuro, y posteriormente con el uso de un epsilon decaimiento y distintos valores para el factor de descuento.

### Resultados de los experimentos

A continuación se van a incluir los resultados de los experimentos para cada entorno de *gymnasium* que se ha utilizado. Se han tenido en cuenta tres métricas a la hora de evaluar los resultados:

- Máxima proporción de recompensa dada por el agente (MPR)
- Número de episodios terminados correctamente (NC)
- Promedio de longitud por episodio terminado correctamente (PLC)

Para el entorno de Frozen Lake, se ha utilizando dos variantes. Una variante es aquella correspondiente al tamaño del mapa de 4x4, y otra es la variante del tamaño de mapa de 8x8.

#### Frozen Lake 4x4

Se presenta en la siguiente tabla los resultados para los que los modelos, de forma global, rindieron mejor. No se va a representar toda la combinatoria de hiperparámetros debido a su extensión.

Tabla 2.1: **Frozen Lake 4x4** -  $\epsilon=0.3$ ,  $df=0.9$

| Agente        | MPR          | NC           | PLC         |
|---------------|--------------|--------------|-------------|
| MC On-Policy  | <b>0.660</b> | <b>33041</b> | <b>7.83</b> |
| MC Off-Policy | 0.658        | 32948        | 7.86        |
| SARSA         | 0.652        | 32641        | 7.96        |
| Q-Learning    | 0.656        | 32815        | 7.89        |

Se puede ver como todos los modelos consiguen resultados similares, si bien para otras combinaciones de parámetros existen diferencias más acentuadas. **Monte Carlo On-Policy**, para este caso concreto, ha sido el que mejor rendimiento ha dado, para  $\epsilon=0.3$ . Para este caso, un factor de descuento del retorno futuro 0.9 ha resultado ser ventajoso con respecto a tenerlo en cuenta en su totalidad (1.0). Esto implica que los valores cercanos a la meta tienen mayor valor, y el valor se diluye más conforme se aleja de la meta, en este entorno. Dado que hemos valorado también la longitud del episodio, este

valor más comedido puede propiciar encontrar caminos más cortos a la meta.

Se presentan los mejores resultados usando  $\epsilon$ -decaimiento.

Tabla 2.2: **Frozen Lake 4x4** -  $\epsilon$ -decaimiento,  $df=0.9$

| Agente        | MPR          | NC           | PLC         |
|---------------|--------------|--------------|-------------|
| MC On-Policy  | 0.896        | 44838        | <b>6.32</b> |
| MC Off-Policy | 0.893        | 44671        | 6.34        |
| SARSA         | <b>0.897</b> | <b>44865</b> | 6.36        |
| Q-Learning    | 0.895        | 44777        | 6.35        |

Utilizando un  $\epsilon$ -decaimiento, podemos comprobar como en general todos los agentes consiguen mejorar sus estadísticas, y esto tiene mucho sentido. Tras una fase inicial exploratoria, donde se tomaron un gran número de acciones aleatorias, con un valor de epsilon alto, los algoritmos se vuelven más greedy (y menos exploratorios) conforme al paso del tiempo y al descenso de epsilon, lo que les lleva a seguir su política óptima, que ya ha aprendido los patrones del entorno.

#### Frozen Lake 8x8

El entorno de Frozen Lake 8x8 es más complejo que el de tamaño 4x4. Al haber mayor número de casillas, también hay mayor número de agujeros y por tanto es más probable que el agente caiga por uno antes de llegar al objetivo. Los mejores resultados, en primer lugar, para un epsilon fijo, se pueden visualizar a continuación.

Tabla 2.3: **Frozen Lake 8x8** -  $\epsilon=0.3$ ,  $df=0.9$

| Agente        | MPR          | NC           | PLC          |
|---------------|--------------|--------------|--------------|
| MC On-Policy  | 0.689        | 34485        | 20.59        |
| MC Off-Policy | 0.330        | 16543        | 20.58        |
| SARSA         | <b>0.758</b> | <b>37907</b> | 19.60        |
| Q-Learning    | 0.536        | 26818        | <b>19.56</b> |

Se puede ver como en general los resultados son peores que para el mapa de 4x4 celdas, lo cual era esperable. Destaca el algoritmo de **SARSA**, que obtiene mejor proporción de recompensa y mayor número de episodios completados correctamente, y **Q-Learning**, que consigue mejor promedio de longitud por episodio completado.

Se presentan los mejores resultados usando  $\epsilon$ -decaimiento.

Tabla 2.4: **Frozen Lake 8x8** -  $\epsilon$ -decaimiento,  $df=0.9$

| Agente        | MPR          | NC           | PLC          |
|---------------|--------------|--------------|--------------|
| MC On-Policy  | 0.883        | 44170        | 17.02        |
| MC Off-Policy | 0.885        | 44255        | 15.33        |
| SARSA         | <b>0.919</b> | <b>45978</b> | <b>14.98</b> |
| Q-Learning    | 0.894        | 44710        | 15.09        |

Al igual que para la configuración de escenario de 4x4, los resultados sobre epsilon decaimiento son bastante mejores, por los motivos que hemos comentado anteriormente. Tenemos que, en este caso, **SARSA** ha logrado las mejores métricas de desempeño de forma absoluta.

## Taxi

El entorno de Taxi presenta un conjunto de recompensas más rico que Frozen Lake, pero también presenta mayor complejidad en cuanto a espacio de estados y acciones que el entorno anterior. En este caso, no existen variantes para el tamaño del entorno.

Se presentan los mejores resultados para valores de epsilon fijo.

Tabla 2.5: **Taxi** -  $\epsilon=0.3$ ,  $df=1.0$

| Agente        | MPR           | NC           | PLC          |
|---------------|---------------|--------------|--------------|
| MC On-Policy  | -127.15       | 36961        | 31.32        |
| MC Off-Policy | -56.28        | 47707        | 25.36        |
| SARSA         | -19.75        | 49740        | 21.41        |
| Q-Learning    | <b>-17.23</b> | <b>49809</b> | <b>20.38</b> |

El agente que mejor ha desempeñado ha sido **Q-Learning**, para un factor de descuento del retorno futuro de 1.0 (no se descuenta el retorno futuro). De nuevo, el valor más ventajoso de  $\epsilon$  es 0.3. Vemos como en este problema destacan sobre todo los agentes que utilizan Diferencias Temporales por encima de Monte Carlo (que calcula los retornos al terminar un episodio). Esto puede ser debido a que existen recompensas intermedias, y que el número de pasos hasta llegar a una solución puede ser generalmente mayor debido a la complejidad que presenta el entorno en cuanto a estados. Los métodos basados en Diferencias Temporales aprovechan mejor cada interacción con el entorno.

Se presentan, por último, los mejores resultados usando  $\epsilon$ -decaimiento.

Tabla 2.6: **Taxi** -  $\epsilon$ -decaimiento,  $df=1.0$

| Agente        | MPR           | NC           | PLC          |
|---------------|---------------|--------------|--------------|
| MC On-Policy  | -166.65       | 24664        | 53.61        |
| MC Off-Policy | -54.30        | 45726        | 18.14        |
| SARSA         | -18.00        | 48829        | 15.48        |
| Q-Learning    | <b>-16.04</b> | <b>48942</b> | <b>15.20</b> |

Vemos que, por lo general, también mejoran con respecto a un valor de  $\epsilon$  fijo. De nuevo, el mejor factor de descuento ha sido 1.0, y el agente que mejores resultados ha ofrecido ha sido **Q-Learning**, seguido de cerca por **SARSA**.

### 2.2.3. Conclusiones

- ▶ La búsqueda de hiperparámetros ha resultado ser efectiva para optimizar el funcionamiento de los agentes:
  - Un factor de descuento de **0.9 del retorno futuro** ha resultado dar mejores resultados que tenerlo plenamente en cuenta (utilizando un valor de 1.0) para el entorno de **FrozenLake**, y tenerlo en cuenta **totalmente** ha resultado mejor para el entorno de **Taxi**.
  - En caso de utilizar  $\epsilon$  fijo durante todo el entrenamiento, ha dado mejor resultado utilizar un valor comedido, como **0.3**, que valores más grandes, como **0.5**. Los agentes se vuelven demasiado aleatorios.

- ▶ El uso de  $\epsilon$ -decaimiento, y que, por tanto, el agente presente al principio un comportamiento más exploratorio, y al final de la ejecución un comportamiento más greedy, explotando la política que ha aprendido, resulta dar también resultados más ventajosos que utilizar un valor fijo para  $\epsilon$ .
- ▶ La evaluación en un entorno de mayores dimensiones y complejidad, como el de Frozen Lake 8x8, dificulta el entrenamiento de los agentes. Son menos susceptibles a encontrar una solución (y por tanto, llegar a aprender una política y mucho menos óptima), e incrementa también el tiempo de cómputo del algoritmo.
- ▶ Por lo general, todos los agentes evaluados han mostrado resultados satisfactorios. No obstante, **Monte Carlo On-Policy** ha demostrado un comportamiento más consistente en el entorno **FrozenLake 4x4**, mientras que **Q-Learning** y **SARSA** han obtenido mejores resultados en **FrozenLake 8x8** y **Taxi**. Estos resultados son coherentes con el teorema No Free Lunch, que establece que no existe un algoritmo óptimo para todos los problemas posibles. En consecuencia, el rendimiento de un agente depende en gran medida de las características del entorno en el que opera, por lo que distintos algoritmos pueden resultar más adecuados para diferentes tareas.

### 2.2.4. Vías futuras

Se exponen las consideraciones y vías futuras para este estudio sobre modelos tabulares.

- ▶ Se podría ampliar la experimentación probando también con valores más grandes para el número de episodios.
- ▶ Sería interesante probar con diferentes valores para el 'learning rate' para SARSA y Q-Learning en el grid de parámetros para ver si mejoran su desempeño con respecto, por ejemplo, a Monte Carlo.
- ▶ Dado que existe la opción de configurar mapas para el Frozen Lake, sería también interesante entrenar a los agentes en uno customizado.
- ▶ Sería también muy interesante, de alguna forma, modificar el entorno para implementar recompensas negativas en los agujeros de Frozen Lake, y ver de qué forma repercute esto en el aprendizaje de los agentes.

## 2.3. Técnicas Aproximadas de Aprendizaje por Refuerzo

A diferencia de los modelos tabulares, que utilizan una tabla que asocia cada posible estado a un valor para cada posible acción en ese estado, los modelos aproximados realizan el aprendizaje de una función de valor  $q(s, a)$ , que puede corresponderse, por ejemplo, con una función lineal, o una red neuronal, y que se asocia con una serie de pesos que pondera a cada característica que identifica a ese estado. De esta forma, dado un estado, es posible determinar que acciones puede realizar el agente, sin tener que almacenar una tabla infinitamente grande que contemplase todas las casuísticas, lo cual no sería posible.

Los modelos aproximados, al igual que los tabulares, presentan una serie de ventajas:

- ▶ Pueden estimar valores para estados nunca vistos a partir de experiencias previas similares, lo que supone que el agente es capaz de generalizar.
- ▶ Son escalables, es decir, permiten trabajar con conjuntos de estados muy grandes o continuos donde una tabla sería inviable.
- ▶ Solo almacenan los parámetros del modelo (pesos), en lugar de una entrada para cada estado o estado-acción, lo cual supone un consumo de memoria generalmente inferior a los modelos tabulares.

También presentan una serie de desventajas con respecto a los modelos tabulares:

- ▶ A diferencia de muchos métodos tabulares, pueden divergir o volverse inestables durante el entrenamiento, con lo que no garantizan su convergencia.
- ▶ Requieren elegir y ajustar modelos, hiperparámetros y técnicas de entrenamiento, lo que complejiza su arquitectura.
- ▶ Al generalizar, una actualización puede afectar a muchos estados simultáneamente, y potencialmente introducir errores en regiones que antes estaban bien estimadas.

### 2.3.1. Algoritmos

Se presentan los algoritmos aproximados que hemos utilizado para los experimentos de esta sección.

- ▶ **SARSA Semi-Gradient:** Es una extensión de SARSA diseñada para entornos con espacios de estados grandes o continuos. En lugar de almacenar una tabla de valores  $Q(s, a)$ , utiliza una función aproximadora parametrizada para estimar dichos valores. La actualización se realiza mediante métodos de descenso por gradiente utilizando el error de diferencias temporales, manteniendo el carácter On-Policy del algoritmo. En nuestro caso, la implementación de SARSA Semi-Gradient se ha realizado con una función de **aproximación lineal**, para la que ha ido aprendiendo y ajustando los pesos. Como este agente no admite un conjunto de estados continuos para trabajar, se ha usado la técnica de **Tile Encoding** para discretizar, mediante una serie de *tiles*, solapados entre sí, el conjunto de estados del entorno, de forma que, dada una posición y una velocidad en un momento determinado, se activen una serie de *tiles* que indiquen qué pesos de la función lineal hay que tener en cuenta.
- ▶ **Deep Q-Learning:** Este método, Off-Policy, queda conformado por dos redes neuronales. Una se utilizará para predecir los retornos de la política de comportamiento y la otra para obtener los de la política objetivo. En cada nuevo paso, se utilizará la red de la política de comportamiento para decidir la acción a realizar, y cuando se llene el replay buffer, es decir, una “memoria” de los pasos que hemos ido dando, se utilizará la red objetivo para calcular los retornos esperados, calcular las diferencias temporales, y actualizar los pesos de la red original. Cada cierta cantidad de pasos se “congela” la red de comportamiento en la red objetivo, en aras de actualizar sus pesos.

### 2.3.2. Wrappers del entorno

Ha sido necesario definir dos *wrappers*, o clases “envoltorio” para modificar algunas partes del entorno debido a algunas exigencias particulares de los algoritmos que hemos utilizado, que reseñamos brevemente:

- ▶ **Wrapper para Tile Enoding:** Se ha definido un wrapper que, utilizando la técnica de Tile Encoding, explicada anteriormente, consiga, dado un estado continuo, discretizarlo y devolver una lista discreta de índices de los tiles que han sido activados. Utilizaremos este wrapper para realizar el entrenamiento en SARSA Semi-gradient.
- ▶ **Wrapper para modificación de recompensa:** Se ha definido también un wrapper que, dada una transición, modificara la recompensa del entorno para que fuera más rica. Originalmente solo se tenía un descuento de -1 por paso de tiempo. Se modificó esto para que se tuviera en cuenta la posición del agente con respecto al objetivo, y su velocidad, dado que con la configuración original de recompensas Deep Q-Learning no lograba encontrar una solución. La fórmula resultante es la siguiente:  $RecompensaModificada = RecompensaOriginal + [posicion + 0,5] + speed^2 * 10$ . Este wrapper es usado para el entorno con el que se entrena Deep Q-Learning.

### 2.3.3. Evaluación/Experimentos

#### Descripción de los experimentos

El objetivo de la experimentación para métodos aproximados ha sido el de establecer una comparativa entre los dos tipos de agentes, utilizando distintos hiperparámetros, de cara a evaluar el desempeño de todos ellos utilizando una vez más la API de *gymnasium*, para un entorno que presenta un conjunto de estados continuo, y hace por tanto imposible la labor de emplear métodos tabulares.

El entorno que se ha elegido es **Mountain Car**, que trata de guiar a un coche, que comienza en la parte baja de un valle, para que suba hasta una colina que se sitúa a su derecha (el objetivo). El espacio de estados queda compuesto por dos componentes continuas, que son la posición y la velocidad del coche. Las posibles acciones son discretas y son acelerar a la izquierda, acelerar a la derecha, y no acelerar. En cuanto a recompensas, se da -1 por cada instante de tiempo, y 0 por llegar al objetivo. El problema no es trivial porque, sencillamente acelerando a la derecha el coche no logra subir la colina. Es necesario acelerar primero a la izquierda para coger inercia y poder subirla.

En cuanto a la metodología de trabajo, se va a entrenar con distintas variantes de los agentes aproximados y diversos hiperparámetros para los mismos. Siendo así, seguiremos los siguientes pasos:

1. Para el entorno, se entrenó los agentes aproximados, utilizando un grid de hiperparámetros, a lo largo de **2000 episodios** para cada combinación (estos agentes tardan bastante más tiempo en ser entrenados).
2. Se obtendrán una serie de estadísticas que nos permitan graficar la recompensa obtenida por cada agente a lo largo del tiempo, y la evolución de la longitud de cada

episodio, en *steps*.

3. Se comparará la mejor recompensa obtenida por el agente con el resto de agentes, y la mejor longitud de episodio que es capaz de conseguir con la del resto de agentes y con la óptima.
4. Se pomediarán los resultados obtenidos para cada entorno, para cada agente.
5. Se darán unas conclusiones finales que indiquen qué algoritmo o algoritmos se considera que presentan un mejor comportamiento.

En este caso, en cuanto al grid de parámetros, se ha decidido trabajar directamente siempre con un  $\epsilon$ -decaimiento, visto en experimentos anteriores que daba mejores resultados en todos los casos. En este caso, se ha planteado utilizar distintos valores para los siguientes parámetros, que nos parecía que podían realizar una mayor aportación en el aprendizaje de este tipo de agentes:

- ▶ **decay\_factor**: Factor de decaimiento para usarlo con  $\epsilon$ -decaimiento. Cuando mayor sea, más decrecerá su valor a lo largo de los episodios.
- ▶ **learning\_rate**: Tasa de aprendizaje para los algoritmos aproximados. Un mayor valor hará que olviden más rápido experiencias pasadas y se centren más en las presentes.
- ▶ **discount\_factor**: Factor de descuento para el retorno futuro, como ya utilizamos para métodos tabulares.

## Resultados de los experimentos

Utilizando esta configuración de grid de parámetros, nos encontramos con que al final todos los agentes acaban aprendiendo y solucionando el problema utilizando una configuración u otra. Sin embargo, nos encontramos con que los valores de tasa de aprendizaje que hacían aprender una solución a SARSA Semi-Gradient eran distintos a los que funcionaban bien para Deep Q-Learning. Cada agente resultó dar por separado los mejores resultados para mayor número de episodios terminados, y menor longitud promedio por episodio terminado, así que se presentan los resultados para cada uno, donde veremos que el otro algoritmo no ha conseguido resolver el problema.

En primer lugar, se puede ver como Deep Q-Learning ha sido el que mayor número de episodios ha completado correctamente.

Tabla 2.7: **Mountain Car** - decf=0.997, df=0.99, lr=0.0005

| Agente              | MPR            | NC          | PLC           |
|---------------------|----------------|-------------|---------------|
| Deep Q-Learning     | <b>-152.49</b> | <b>1699</b> | <b>144.07</b> |
| SARSA Semi-gradient | -200           | 0           | 0             |

Vemos como estos valores se obtienen para un factor de decaimiento de 0.997, un factor de descuento de 0.99 y una tasa de aprendizaje de 0.0005. Al parecer, tasas de aprendizajes más pequeñas favorecen a la red neuronal de Deep Q-Learning. En las gráficas se pudo apreciar como la longitud promedio por episodio tenía momentos en los que subía repentinamente para luego volver a bajar, lo que penalizaría esta métrica para este agente.

Se muestran los mejores resultados en cuanto a longitud promedio de episodio completado, así como proporción de recompensa.

Tabla 2.8: **Mountain Car** - decf=0.997, df=0.99, lr=0.05

| Agente              | MPR            | NC          | PLC           |
|---------------------|----------------|-------------|---------------|
| Deep Q-Learning     | -200           | 0           | 0             |
| SARSA Semi-gradient | <b>-145.57</b> | <b>1570</b> | <b>130.67</b> |

Los mejores valores para factor de decaimiento y factor de descuento son similares a los del caso anterior, pero, en este caso, la tasa de aprendizaje es de 0.05. Esto da a entender que tasas de aprendizaje más grandes favorecen al funcionamiento de SARSA Semi-gradient, en comparación.

## 2.3.4. Conclusiones

- ▶ **SARSA Semi-gradient**, si bien ha sido el agente más sencillo en cuanto a funcionamiento e implementación, también ha mostrado ser muy efectivo en la resolución de este problema, funcionando de forma más consistente que Deep Q-Learning, al menos en cuanto a reducción de la longitud de los episodios se refiere. Al ser más consistente, también nos ha dado un promedio de longitud por episodio completado menor, de 130 pasos en promedio. Ha requerido discretizar el espacio de estados utilizando la técnica de Tile Encoding.
- ▶ **Deep Q-Learning** es más complejo que SARSA y lento de entrenar, dado que utiliza dos redes neuronales. También presenta una serie de ventajas, como que no requiere discretizar el espacio de estados, sino que es capaz de trabajar con variables continuas, pero es por contra mucho más sensible, por ejemplo a los valores de las recompensas. Debido a esto, ha sido necesario declarar un wrapper que modifique la recompensa original del entorno por otra que tuviera en cuenta las componentes de posición y velocidad. En esencia, este modelo ha demostrado tener una potencia considerable, logrando identificar como resolver el problema en apenas 301 episodios, para solucionarlo correctamente en otros 1699, como hemos mostrado en los resultados globales.
- ▶ La búsqueda de hiperparámetros ha resultado ser efectiva para optimizar el funcionamiento de los agentes:
  - El valor para la tasa de aprendizaje se ha mostrado como crítico para que el agente logre resolver o no resolver el problema. Valores pequeños, como 0.0005, son mucho más acordes para una red neuronal, y valores más grandes, como 0.05, funcionan mucho mejor para SARSA Semi-Gradient. Este es un parámetro determinante y es aconsejable dedicar tiempo a buscar el valor óptimo para cada agente y problema.
  - Un factor de descuento del retorno futuro de 0.99 también ha resultado ser más adecuado en este problema que un factor de 0.9, lo que significa que, para este entorno en particular, interesa más que en agente tenga en cuenta el futuro a medio plazo que el futuro inmediato de cara a realizar el aprendizaje.
  - También se ha podido comprobar como un factor de decaimiento moderado, como 0.997, funciona mejor que un factor de 0.999 para epsilon decay, haciendo que epsilon decaiga de manera más acentuada confor-

me al transcurso de los episodios, y se vaya reduciendo por tanto antes la capacidad exploratoria del agente.

- ▶ La designación de un conjunto rico de recompensas, en este caso, para **Deep Q-learning**, ha resultado una vez más mostrarse como positiva y recomendable cuando se tiene un entorno que, si bien no sea trivial, presente cierta dificultad como el que tenemos entre manos.

### 2.3.5. Vías futuras

Se exponen las consideraciones y vías futuras para este estudio sobre modelos aproximados.

- ▶ Se podría ampliar el número de episodios en el que se entrenan los agentes para comprobar cómo evolucionan y si cambian los resultados globales que se obtienen.
- ▶ Se podría probar con la variante de este entorno, **MountainCarContinuous** que presenta, no solo un conjunto de estados compuesto por componentes continuas, sino un conjunto de acciones continuo. Sería interesante ver cómo se desempeñan SARSA Semi-gradient (discretizando también acciones) y Deep Q-Learning en ese entorno, que seguramente presentará una dificultad añadida.
- ▶ Sería interesante investigar más a fondo ese incremento en la longitud del episodio que ha experimentado Deep Q-Learning hacia el final del entrenamiento, para ver por qué se ha producido e intentar mitigarlo.

# Bibliografía

- [1] Hernández Roldán, Iván and Magarzo, Gonzalo (2022). *Contextual Bandits: Foundations and Applications*. Facultad de Informática, Universidad Complutense de Madrid
- [2] Documentación sobre API de gymnasium. <https://gymnasium.farama.org/index.html>
- [3] Las transparencias de la asignatura.
- [4] Uso de la IA Generativa:
  - 1. (K-Brazos) Se ha pedido un método para promediar varios sets de resultados, que nos permite identificar el mejor algoritmo globalmente.
  - 2. (K-Brazos) Se ha preguntado qué valores de  $c$  son adecuados para experimentación con UCB-1 y qué valores de temperatura lo son para Softmax.
  - 3. (K-Brazos) Se ha pedido indagar un poco más en el funcionamiento de Softmax porque se quería entender los resultados pobres que ha dado en este experimento.
  - 4. (Entornos Complejos) Se ha pedido la implementación de la gráfica utilizando una ventana deslizante configurable para la longitud por episodio completado correctamente.
  - 5. (Entornos Complejos) Se ha indagado más en la documentación de gymnasium, en este caso, del entorno de Frozen Lake, para ver que implicaciones podría haber tenido usar parámetros como `slipery=True` en cuanto al comportamiento del mismo. Se pidieron formas de representar tabularmente la tabla Q que se construye para que fuera legible y más visual, pero al final no se llegaron a utilizar.
  - 6. (Entornos Complejos) Se intentó también hacer uso de la IA generativa para entender mejor por qué los modelos basados en diferencias temporales podían funcionar mejor para el entorno Taxi que con Monte Carlo.
  - 7. (Entornos Complejos) Si bien una primera modificación del wrapper que modificaba las recompensas (para ser utilizado por Deep-QLearning, en el entorno de Mountain Car) fue propia, concretamente tener en cuenta la posición, la IA Generativa sugirió la utilización de la segunda componente con la velocidad, con la que aprende mucho mejor.
  - 8. (Entornos Complejos) Fue utilizada también para sugerir valores para la tasa de aprendizaje que pudieran dar buenos resultados para las redes neuronales.
  - 9. (Entornos Complejos) Dado que la asignatura de Deep Learning aún no la he cursado, he utilizado la IA para que me explicara de forma sencilla como funciona una red neuronal, de cara a comprender mejor el funcionamiento de Deep Q-Learning previo a su implementación (si bien la clase grabada fue muy esclarecedora).